

RAPPORT TECHNIQUE APPROFONDI — APPLICATION JEU

Explication détaillée : briques graphiques Swing & logique métier commentée

Ce document complète le rapport d'architecture. Il a un but **pédagogique** : expliquer **comment fonctionne** chaque brique du code et **pourquoi** ces choix ont été faits. Il se lit en deux parties : 1. **La boîte à outils graphique** (Swing / AWT) : à quoi sert chaque composant et chaque méthode d'affichage utilisés. 2. **La logique métier, méthode par méthode** : le fonctionnement réel du code, structure de données par structure de données.

PARTIE 1 — LA BOÎTE À OUTILS GRAPHIQUE (SWING / AWT)

L'interface utilise **Swing** (composants `J...`) posé sur **AWT** (dessin, événements, couleurs). Voici chaque brique employée.

1.1 — Le couple `JFrame` / `JPanel`

- **`JFrame`** est une **fenêtre** du système d'exploitation (barre de titre, bordures, croix de fermeture). Nos fenêtres en **héritent** (`class FrameAccueil extends JFrame`).
- **`JPanel`** est un **conteneur léger** sans décoration, sur lequel on pose les composants et on dessine. Nos panneaux en héritent (`class PanelGrille extends JPanel`).

Pourquoi séparer Frame et Panel ? C'est la convention MVC/Swing : la `Frame` n'est qu'une coquille, tout le contenu vit dans un `Panel` réutilisable et personnalisable. Une fenêtre ne contient quasiment que `this.add(new PanelXxx(...))`.

Méthodes de configuration de fenêtre utilisées : - **`setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`** : cliquer la croix **arrête le programme**. - **`pack()`** : ajuste la fenêtre à la **taille préférée** de son contenu (utilisé quand la taille dépend du plateau). À l'inverse, **`setSize(l, h)`** impose une taille fixe (accueil). - **`setLocationRelativeTo(null)`** : **centre** la fenêtre à l'écran. - **`setVisible(true/false)`** : affiche ou masque. On **masque** souvent l'écran précédent (`frmAccueil.setVisible(false)`) au lieu de le détruire, pour pouvoir y revenir.

1.2 — Les gestionnaires de disposition (`LayoutManager`)

On ne place presque jamais les composants en pixels : on confie leur disposition à un **layout**, qui réagit aux redimensionnements. Chaque layout a sa logique :

- **`BorderLayout`** — 5 zones : NORTH, SOUTH, EAST, WEST, CENTER. Le centre **s'étire**. Utilisé dans `PanelJoueur` (grille au centre), `PanelChargement` (formulaire au centre), `PanelResultats` (titre en haut, tableau au centre, gagnant + bouton en bas). **Pourquoi** : idéal dès qu'on a un élément principal central entouré de bandeaux.

- **FlowLayout** — aligne les composants **à la suite**, comme des mots dans une ligne. Utilisé par `FrameChargement`. **Pourquoi** : le plus simple quand on a peu d'éléments.
- **GridLayout(lignes, colonnes, hgap, vgap)** — une grille de cellules de **taille égale**. Utilisé pour le bloc de boutons d'accueil (2,1), le tableau de scores, les blocs de boutons. **Pourquoi** : répartition parfaitement uniforme, parfaite pour des boutons ou un tableau régulier.
- **GridBagLayout** — le plus puissant et le plus souple. Dans `PanelAccueil` il est utilisé **vide** (sans contraintes) : un `GridBagLayout` qui ne contient qu'un seul composant le **centre automatiquement**, horizontalement ET verticalement. **Pourquoi** : astuce simple et fiable pour centrer parfaitement le bloc de boutons par-dessus l'image de fond.
- **BoxLayout(this, BoxLayout.Y_AXIS)** — **empile verticalement** des composants de hauteurs différentes. Utilisé par `PanelInfos` (la bande centrale). **Pourquoi** : une colonne d'éléments hétérogènes (titre, image de carte, labels, boutons) les uns au-dessus des autres.

1.3 — `Box.createVerticalStrut(n)` et `Box.createVerticalGlue()`

Un `BoxLayout` **colle** les composants les uns aux autres, sans espace (il n'a pas de `hgap/vgap` comme `GridLayout`). Pour **aérer**, on insère des « **ressorts** » invisibles :

```
this.add(lblTitre);
this.add(Box.createVerticalStrut(6)); // 6 px d'espace FIXE
this.add(this.lblManche);
...
this.add(Box.createVerticalGlue()); // ressort ELASTIQUE
this.add(this.btnQuitter); // -> poussé tout en bas
```

- **`createVerticalStrut(n)`** crée une **cale rigide** de `n` pixels de haut : un espacement **fixe** entre deux composants. C'est ce qui donne la respiration entre le titre, la manche, la carte, etc.
- **`createVerticalGlue()`** crée un espace **élastique** qui **absorbe tout l'espace libre** restant dans la colonne. Placé juste avant le bouton « Quitter », il **pousse ce bouton tout en bas** de la bande, quelle que soit la hauteur de la fenêtre.

Pourquoi ces deux outils ? C'est la manière **idiomatique** d'espacer dans un `BoxLayout`. Retenir : **Strut = marge fixe, Glue = « pousser » / occuper le vide**.

Compléments utilisés dans la même bande : - **`setAlignmentX(Component.CENTER_ALIGNMENT)`** : dans un `BoxLayout` vertical, un composant tend à se caler à gauche ; cette ligne le **centre horizontalement**. - **`setMaximumSize(new Dimension(180, 26))`** : par défaut un composant s'étire en largeur sur toute la bande ; on **borne sa largeur** pour garder des boutons de taille raisonnable.

1.4 — Le dessin personnalisé : `paintComponent(Graphics g)`

Quand on veut dessiner soi-même (l'image de fond, la grille, le tracé du réseau), on **redéfinit** `paintComponent` :

```
protected void paintComponent(Graphics g)
{
    super.paintComponent(g); // 1. laisse Swing nettoyer le fond
    g.drawImage(imgFond, 0, 0, getWidth(), getHeight(), this); // 2. notre dessin
}
```

- `super.paintComponent(g)` en premier est **obligatoire** : il laisse Swing effacer l'ancien rendu avant le nôtre, sinon on accumule des artefacts visuels.
- L'objet `Graphics g` est le « **pinceau** ». Méthodes employées :
- `drawImage(img, x, y, w, h, this)` : dessine une image **étirée** aux dimensions voulues (fond, stations, carte).
- `setColor(c)` puis `fillRect(x, y, w, h)` : remplit un rectangle (fond des cases, surbrillance, voile sombre).
- `drawLine(x1, y1, x2, y2)` : trace un segment (la ligne du réseau entre deux stations).
- `drawOval(x, y, w, h)` : dessine un cercle (l'anneau autour de la case de départ).
- **La transparence (canal alpha)** : `new Color(r, g, b, a)` accepte un **4e paramètre a** (0 à 255) = l'opacité. Exemples : `new Color(0, 0, 0, 130)` = noir à ~50 % pour le **voile** de lisibilité de l'écran de chargement ; `new Color(255, 255, 0, 120)` = **jaune translucide** pour la surbrillance des cases jouables, qui laisse voir la couleur d'arrondissement en dessous. **Pourquoi** : superposer des calques tout en gardant la lisibilité de ce qu'il y a dessous.
- On ne dessine **jamais** directement : on appelle `repaint()` qui demande à Swing de **replanifier** un `paintComponent` au bon moment.

1.5 — Les images : `ImageIcon` / `Image` et le cache

- `new ImageIcon(chemin).getImage()` charge un fichier image (PNG) en mémoire.
- Le contrôleur garde des **caches** : `imagesStations[]` et `imagesCartes[][]`. Le chargement est **paresseux (lazy)** : une image n'est lue sur le disque qu'à la **première demande**, puis **réutilisée**.

```
public Image getImageStation(int numero)
{
    if (this.imagesStations[numero] == null)           // pas encore chargée ?
        this.imagesStations[numero] = new ImageIcon(...).getImage(); // on la charge 1 fois
    return this.imagesStations[numero];                 // sinon on réutilise
}
```

Pourquoi un cache ? `paintComponent` est rappelé très souvent (à chaque `repaint`). Relire l'image sur le disque à chaque fois serait **lent**. Le cache échange un peu de mémoire contre beaucoup de vitesse — un compromis classique.

1.6 — Les événements : `ActionListener` et `MouseListener`

- **`ActionListener` / `actionPerformed(ActionEvent e)`** : déclenché par un **clic sur un bouton** ou une sélection. On identifie la source de deux façons :
- `e.getSource() == this.btnJouer` : comparaison directe à la référence du bouton.
- `e.getActionCommand()` : on lit une **étiquette** posée sur le bouton. Dans `PanelInfos`, les boutons « Passer » sont créés **en boucle** (un par joueur) ; plutôt que de garder N références, on étiquette chacun (`setActionCommand("PASSER_" + j)`) et on relit l'étiquette pour retrouver le joueur. **Pourquoi** : méthode élégante pour des composants **générés dynamiquement**.
- **`MouseListener` / `mousePressed(MouseEvent e)`** : capte les **clics souris** sur la grille. `e.getX()` / `e.getY()` donnent la position en **pixels**, qu'on convertit en **numéro de case** (voir `PanelGrille`).

- Les méthodes vides (`mouseClicked`, `mouseReleased`, `mouseEntered`, `mouseExited`) : `MouseListener` est une **interface**, donc on **doit** implémenter toutes ses méthodes, même celles qu'on n'utilise pas — d'où les corps {}.

1.7 — `JOptionPane`

```
JOptionPane.showMessageDialog(null, texte);
```

- `JOptionPane.showMessageDialog` ouvre une petite **boîte de dialogue modale** : un message + un bouton **OK**. Le premier argument `null` signifie **pas de fenêtre parente** (la boîte se centre à l'écran).
- **Où** : la méthode `afficherScoresManche()` du contrôleur l'utilise pour annoncer « **Manche X terminée** » suivi des scores.
- **Pourquoi ce choix ?** C'est le moyen le plus simple d'**interrompre** le joueur pour une information importante, **sans construire une fenêtre dédiée**. « Modale » veut dire que le jeu **attend** que l'utilisateur clique OK avant de continuer : c'est exactement l'effet voulu pour marquer une **pause nette entre deux manches**.

1.8 — Les composants de saisie

- `JComboBox<String>` : liste déroulante (choix du fichier de plateau, et en debug le type de carte). `addItem(...)` la remplit, `getSelectedItem()` / `getSelectedIndex()` lisent le choix.
- `JRadioButton + ButtonGroup` : choix **exclusif**. `rbLocal` et `rbMulti` sont mis dans un même `ButtonGroup`, qui garantit qu'**un seul** peut être coché à la fois.
- `JTextField` : champ texte (nombre de manches). `getText()` renvoie une **chaîne**, qu'on convertit par `Integer.parseInt(...)` à l'intérieur d'un `try/catch` — c'est la **gestion d'erreur de saisie** (si l'utilisateur tape « abc »).
- `JLabel` : texte non éditable (statuts, titres). `setText(...)` le met à jour à chaque rafraîchissement.
- `JSeparator` : une simple **ligne horizontale** de séparation visuelle dans la bande.

1.9 — Le placement multi-fenêtres : `Toolkit` & `setBounds`

```
Dimension ecran = Toolkit.getDefaultToolkit().getScreenSize();  
this.frameInfos.setBounds(bx, 20, bandeW, bandeH);
```

- `Toolkit.getDefaultToolkit().getScreenSize()` récupère la **résolution de l'écran**.
- `setBounds(x, y, largeur, hauteur)` positionne une fenêtre en **pixels absolus**.
- **Pourquoi en pixels ici (et pas un layout) ?** Un `LayoutManager` organise l'intérieur d'une fenêtre. Ici on veut disposer **plusieurs fenêtres** sur l'écran (la bande centrale au milieu, les plateaux des joueurs de part et d'autre) : c'est un agencement **à l'échelle de l'écran**, qui se calcule donc en pixels à partir de `getScreenSize()`.

1.10 — `Desktop` : ouvrir le PDF des règles

```
if (fichierPdf != null && Desktop.isDesktopSupported())
    Desktop.getDesktop().open(fichierPdf);
```

- `Desktop.getDesktop().open(fichier)` demande au **système d'exploitation** d'ouvrir un fichier avec son application par défaut (ici, le PDF des règles avec le lecteur PDF de l'utilisateur).
- `Desktop.isDesktopSupported()` vérifie d'abord que la fonctionnalité est disponible sur la plateforme, pour éviter une erreur.

PARTIE 2 — LA LOGIQUE MÉTIER, MÉTHODE PAR MÉTHODE

Le paquet `plateau.metier` ne contient **aucun** code graphique. C'est le **modèle** pur : structures de données et règles.

Plateau — le support de données

Le plateau mémorise tout sous forme de **tableaux d'entiers à une dimension**, indexés par **numéro de case** (de 0 à `largeur*hauteur-1`) :

```
private int[] tabArrondissements; // l'arrondissement de chaque case
private int[] tabStations;        // le type de station (0 = aucune)
private int[] tabDeparts;         // le n° de joueur dont c'est le départ (0 = aucun)
```

Pourquoi un tableau 1D et pas une grille 2D ? Parce qu'on passe facilement de l'un à l'autre : la case `i` est à la **colonne** `i % largeur` et à la **ligne** `i / largeur`. Un tableau 1D est plus simple à parcourir (une seule boucle) et à sauvegarder ligne par ligne. Tous les accesseurs (`getStation`, `affecterStation...`) **vérifient les bornes** (`numCase >= 0 && numCase < length`) pour ne jamais planter sur un index invalide.

GraphePlateau — le réseau de liaisons

Le réseau de métro est un **graphe non orienté**, stocké en **matrice d'adjacence** :

```
private boolean[][] matriceAretes; // matriceAretes[i][j] == true <=> i et j sont reliées
```

- `ajouterArete(i, j)` pose l'arête **dans les deux sens** (`[i][j]` ET `[j][i]`). **Pourquoi ?** Le graphe est **non orienté** : si A est relié à B, alors B est relié à A. La matrice est donc **symétrique**.
- `aArete(i, j)` : simple lecture de la case `[i][j]`.

genererAretesAuto(plateau) — génération par lancer de rayon

```
int[] déplacementsX = {1, 1, 0, -1}; // Est, Sud-Est, Sud, Sud-Ouest
int[] déplacementsY = {0, 1, 1, 1};
for (int i = 0; i < taille; i++) {
    if (plateau.getStation(i) == 0) continue; // on ne part que des stations
    int x = i % largeur, y = i / largeur;
    for (int direction = 0; direction < 4; direction++) {
        int cx = x + déplacementsX[direction];
        int cy = y + déplacementsY[direction];
```

```

        while (/* dans la grille */) {
            int cible = cy * largeur + cx;
            if (plateau.getStation(cible) > 0) { ajouterArete(i, cible); break; } // 1re station -> stop
            cx += déplacementsX[direction];
            cy += déplacementsY[direction];
        }
    }
}

```

Comment ça marche : pour chaque station, on **part dans 4 directions** (Est, Sud-Est, Sud, Sud-Ouest) et on **avance case par case** jusqu'à rencontrer la **première** station ; on la relie, puis on **s'arrête** (`break`). C'est un **lancer de rayon** (*ray-casting*).

Pourquoi seulement 4 directions et pas 8 ? Parce que les arêtes sont **symétriques** : le rayon « Est » d'une station est exactement le rayon « Ouest » de sa voisine. Couvrir Est/Sud-Est/Sud/Sud-Ouest pour **toutes** les stations suffit à créer les 8 directions sans doublon.

Pourquoi s'arrêter à la première station (`break`) ? Pour que deux stations soient reliées **seulement si rien ne s'intercale** entre elles : une station joue le rôle de « mur » qui coupe la ligne de visée. C'est la règle de connexion du jeu.

Carte — une carte de la pioche

Classe **simple et immuable** : un type de station (1..N, ou la constante `JOKER = 0`) et un booléen `estFoncee`. **Pourquoi une constante `JOKER = 0` ?** Plutôt qu'un type spécial, on réserve la valeur 0 ; la méthode `estJoker()` teste juste `typeStation == 0`. C'est léger et lisible.

Pioche — le paquet commun

```

this.cartes = new ArrayList<Carte>();
for (int type = 1; type <= nbStations; type++) cartes.add(new Carte(type, true)); // foncées
cartes.add(new Carte(Carte.JOKER, true)); // joker foncé
for (int type = 1; type <= nbStations; type++) cartes.add(new Carte(type, false)); // claires
cartes.add(new Carte(Carte.JOKER, false)); // joker clair

```

Composition du paquet : une carte **foncée** par type + un **joker foncé**, puis autant de **claires** + un **joker clair**. Le compteur `nbFonceesRestantes` est initialisé à `nbStations + 1`.

Pourquoi compter les foncées séparément ? Parce que la **fin de manche** ne dépend pas de l'épuisement total du paquet, mais du fait que **toutes les cartes foncées ont été tirées** (`estTerminee()` renvoie `nbFonceesRestantes <= 0`). Les foncées agissent comme un **compte à rebours**.

melanger() — le brassage de Fisher-Yates

```

Random rng = new Random();
for (int i = cartes.size() - 1; i > 0; i--) {
    int j = rng.nextInt(i + 1); // un indice au hasard entre 0 et i
    Carte tmp = cartes.get(i); // échange i <-> j
    cartes.set(i, cartes.get(j));
    cartes.set(j, tmp);
}

```

Comment ça marche : on parcourt le paquet **de la fin vers le début** et on échange chaque carte avec une carte tirée au hasard **parmi celles non encore fixées**. Un **seul passage**, donc une complexité **linéaire $O(n)$** .

Pourquoi cet algorithme (et pas un tri aléatoire) ? Fisher-Yates est **mathématiquement équitable** : chacune des **$N!$** permutations possibles du paquet a **exactement la même probabilité** d'apparaître, sans biais. C'est la méthode de référence pour mélanger.

piocher()

```
Carte c = cartes.remove(0);           // retire la carte du dessus
if (c.estFoncee()) nbFonceesRestantes--; // met à jour le compte à rebours
return c;
```

Renvoie null si le paquet est vide. **Pourquoi remove(0) ?** On modélise le « dessus » de la pioche par le **début** de la liste ; chaque tirage le retire.

ReseauJoueur — la ligne du joueur (à deux bouts)

La ligne est une **liste ordonnée** de stations (ArrayList<Integer>). Sa particularité : on peut l'agrandir **par les deux extrémités** :

```
public void ajouterStation(int numCase) { stations.add(numCase); } // ajoute à la FIN
public void ajouterDebut(int numCase) { stations.add(0, numCase); } // insère au DEBUT
public int getPremiereStation() { return stations.get(0); } // 1re extrémité
public int getDerniereStation() { return stations.get(stations.size()-1); } // 2e extrémité
```

Pourquoi deux méthodes d'ajout ? Une ligne de métro a **deux bouts** ; le joueur doit pouvoir la prolonger d'un côté **comme** de l'autre. La **première** et la **dernière** station de la liste représentent ces deux extrémités. `contient()` empêche les doublons.

Joueur

Un joueur agrège : son `resseau`, un drapeau `aJoue` (verrou d'une **seule action par carte**), le tableau `scoresManches[]` et le `scoreTotal`. **Pourquoi le drapeau aJoue ?** La carte commune est révélée une fois, et **chaque joueur ne joue qu'un coup** dessus ; `aJoue` interdit de rejouer tant que la carte suivante n'est pas révélée. `getMeilleurScoreManche()` sert au **départage** en cas d'égalité.

ValideurMouvement — les 4 règles dans l'ordre

```
public static boolean estValide(int numCase, Joueur j, Carte carte, Plateau p) {
    if (p.getStation(numCase) == 0) return false; // 1
    if (!carte.estJoker() && p.getStation(numCase) != carte.getTypeStation()) // 2
        return false;
    if (j.getResseau().contient(numCase)) return false; // 3
    int prem = j.getResseau().getPremiereStation();
    int der = j.getResseau().getDerniereStation();
    if (prem == -1) return false;
    return p.getGraphe().aArete(der, numCase) || p.getGraphe().aArete(prem, numCase); // 4
}
```


Comment ça marche, règle par règle : 1. **Station présente ?** Une case vide (`getStation == 0`) est refusée d'emblée. 2. **Bon type ?** Le type de la station doit correspondre à la carte — **sauf** si c'est un **joker** (`estJoker()` court-circuite le test). 3. **Pas déjà prise ?** La case ne doit pas être déjà dans le réseau du joueur (`contient`). 4. **Reliée à un bout ?** La case doit avoir une **arête** vers la **première** OU la **dernière** station (les deux extrémités).

Pourquoi cet ordre précis ? Les tests vont du **moins coûteux / plus fondamental** au plus spécifique, et **chacun sort immédiatement** (`return false`). Conséquence importante pour la démo : si le type est mauvais (règle 2), c'est ce message qui apparaît **avant** la règle 3 ou 4 — l'ordre des `if` détermine la raison affichée.

`getCasesValides(...)` applique `estValide` à **toutes** les cases et renvoie la liste de celles qui passent : c'est exactement ce que l'IHM surligne en **jaune**.

Partie — le chef d'orchestre

La `Partie` tient l'état complet : joueurs, pioche, carte commune, joueur courant, numéro de manche, et les drapeaux `entreManche` / `partieTerminee`.

Détection automatique de la configuration (constructeur)

```
for (int i = 0; i < taille; i++) {
    if (p.getDepart(i) > maxDepart) maxDepart = p.getDepart(i);
    if (p.getStation(i) > maxStation) maxStation = p.getStation(i);
}
this.nbJoueurs = maxDepart; // le nb de joueurs = le plus grand n° de départ
this.nbStations = maxStation; // le nb de types = le plus grand n° de station
```

Pourquoi ? Le nombre de joueurs et de types n'est pas stocké explicitement : on le **déduit** du plateau (le plus grand numéro de départ / de station rencontré). Le plateau est ainsi **auto-descriptif**.

`getCaseDepart(numeroJoueur)` — la rotation des départs

```
int k = ((numeroJoueur - 1) + (numeroManche - 1)) % nbJoueurs;
return casesDepart[k];
```

Comment ça marche : à chaque manche, on **décale** le départ attribué à chaque joueur (arithmétique **modulo** le nombre de joueurs). **Pourquoi ?** Pour l'**équité** : personne ne garde une position de départ avantageuse sur toute la partie. Les **couleurs** tournent avec la **même formule** côté contrôleur.

`jouerCoup(...)` — poser sur le bon bout

```
int derniere = joueur.getReseau().getDerniereStation();
if (this.plateau.getGraphe().aArete(derniere, numCase))
    joueur.getReseau().ajouterStation(numCase); // se rattache par la FIN
else
    joueur.getReseau().ajouterDebut(numCase); // sinon par le DEBUT
```

Comment ça marche : après avoir validé le coup, on regarde si la nouvelle station est voisine de la **dernière** extrémité : si oui on prolonge **par la fin**, sinon c'est qu'elle touche la **première** extrémité et on prolonge **par le début**. **Pourquoi ?** Pour garder la liste **dans l'ordre réel** du tracé, quel que soit le bout par lequel le joueur étend sa ligne.

avancerJoueur() — la mécanique du tour

On cherche le prochain joueur qui n'a pas encore joué. **Quand tous ont joué** la carte commune : - si la pioche est **terminée** (toutes les foncées tirées) -> **finManche()** ; - sinon -> on **révèle la carte suivante**, on remet tous les aJoue à false, et on redonne la main au joueur 1.

finManche() puis continuerManche()

finManche() calcule le score de chaque joueur via CalculateurScore, puis : s'il reste des manches -> passe en attente (entreManche = true) ; sinon -> partieTerminee = true. continuerManche() enchaîne la manche suivante (réseaux remis au départ, départs tournés). **Pourquoi cette pause entreManche ?** Pour laisser le contrôleur **afficher les scores** (le JOptionPane) **avant** de réinitialiser le plateau.

CalculateurScore — le score multiplicatif

```
int[] compteParZone = new int[21]; // stations par arrondissement
for (int s : reseau.getStations())
    compteParZone[ plateau.getArrondissement(s) ]++;
int nbZones = 0, maxDansUneZone = 0;
for (int a = 1; a <= 20; a++) {
    if (compteParZone[a] > 0) nbZones++;
    if (compteParZone[a] > maxDansUneZone) maxDansUneZone = compteParZone[a];
}
return nbZones * maxDansUneZone;
```

Comment ça marche : on **compte les stations par arrondissement**, puis on multiplie le **nombre de zones traversées** par le **plus gros paquet** de stations dans une seule zone.

Pourquoi une multiplication (et pas une somme) ? Pour **forcer un compromis stratégique** : marquer beaucoup de points demande à la fois de l'**étendue** (traverser plusieurs arrondissements -> facteur de gauche) **et** de la **densité** (concentrer des stations dans un même arrondissement -> facteur de droite). Une seule des deux qualités donne un score faible.

ChargeurPlateau — la lecture robuste du fichier

charger(File) lit l'en-tête largeur;hauteur, puis les lignes de cases
index;arrondissement;station;depart, puis d'éventuelles arêtes source;destination.

Comment c'est sécurisé : la méthode renvoie **null** au moindre défaut — fichier absent/illisible, en-tête != 2 valeurs, dimensions hors bornes (<=0 ou >100), index non séquentiel, valeurs hors plage (arrondissement 0–20, station 0–10, départ 0–4), nombre de cases incorrect — et **attrape** NumberFormatException si une valeur n'est pas un nombre. **Pourquoi tant de gardes ?** Un fichier édité à la main peut être corrompu ; on **refuse proprement** plutôt que de planter, et l'IHM affiche alors « fichier invalide ». Si le fichier ne contient **pas** d'arêtes, on les **régénère** automatiquement (genererAretesAuto).

UtilitaireJeu — la boîte à outils

Méthodes **statiques** (pas besoin d'instance) : getCouleurs() (les 20 couleurs d'arrondissements), getCheminImageCarte(...) / getCheminImageStation(...) qui **cherchent une image dans plusieurs emplacements** possibles (selon d'où le programme est lancé). **Pourquoi statique ?** Ce sont des

données et chemins constants, partagés par toute l'application : inutile de créer un objet pour y accéder.

Fin du rapport approfondi — Application Jeu.